

Tema 1: Clases y orientación a objetos

1.1 Repaso: structs y paso de parámetros

Una **struct** agrupa diversos datos relacionados.

```
cpp [main.cpp] struct Reloj {      int h, m, s; }; // Es necesario poner el ';'
```

Paso de parámetros a funciones

- **Por valor:** Se realiza una copia. Lento y no modifica el original.
- **Por referencia (&):**
 - **Lectura (const + &):** Rápido y seguro. No hay copia. Ejemplo: `void mostrar(const Reloj& r);`
 - **Escritura (&):** Modifica el original directamente. `void avanzar(Reloj& r);`

1.2 Tipos abstractos de datos (TAD) y clases

Un **TAD** agrupa datos (atributos) y las operaciones (métodos) para manipularlos. En C++, se implementa mediante **clases** para garantizar la consistencia de los datos.

Encapsulación

Protege los datos para evitar modificaciones externas incontroladas: - **private:** Atributos ocultos al código externo. Accesibles solo por los métodos de la misma clase. - **public:** Métodos accesibles desde fuera. Son la interfaz o única vía válida para interactuar con el objeto.

Organización en ficheros

Separamos el “qué hace” del “cómo lo hace”.

1. Especificación (.hpp o .hh)

Define la estructura de la clase e incluye guardas (**#ifndef**) para evitar inclusiones múltiples y bucles de compilación.

```
“cpp [Punto.hpp] #ifndef PUNTO_HPP #define PUNTO_HPP

class Punto { double x, y; // Privado por defecto

public: Punto(); // Constructor por defecto Punto(double a, double b); // Constructor con parámetros
Punto(const Punto& otro); // Constructor de copia

void mover(double dx, double dy); // Modificador
double get_x() const;           // Consultor (const = no modifica)

}; #endif
```

2. Implementación (`.cpp` o `.cc`)

Contiene el código real de las funciones declaradas.

- **``#include "Punto.hpp"``**: Carga las definiciones de datos previas.

- **``\*Operador `::`\*``** (Resolución de ámbito): Indica a qué clase pertenece la función (ej: `Punto::mover``)

```cpp [Punto.cpp]

#include "Punto.hpp"

Punto::Punto() { x = 0; y = 0; }

Punto::Punto(double a, double b) { x = a; y = b; }

Punto::Punto(const Punto& otro) { x = otro.x; y = otro.y; }

void Punto::mover(double dx, double dy) {

    x += dx; y += dy; // Acceso a los atributos del objeto actual  
}

double Punto::get\_x() const { return x; }

### 3. Uso (main.cc)

Creación y uso de objetos.

“`cpp [main.cpp] #include #include “Punto.hpp”

int main() { Punto p(1, 2); // Llamada al constructor con parámetros p.mover(3, 3); // ‘p’ actúa como parámetro implícito

// std::cout << p.x; // ¡ERROR! ‘x’ es privado

std::cout << p.get\_x(); // Correcto

}

\*[Simulación interactiva disponible a la web]\*

---

### ## 1.3 Conceptos adicionales de clases

- **``Parámetro implícito (`this`)``**: Es un puntero interno oculto que referencia al objeto sobre el cual

- **``Métodos `inline```**: Se escriben directamente dentro del `.hpp`. El compilador sustituye la llamada a

- **``Miembros `static```**:

    - **``Atributos``**: Se comparten globalmente entre todos los objetos de la misma clase. (ej: un contador c

    - **``Métodos``**: No requieren instanciar ningún objeto para ejecutarse. Se llaman mediante la referencia

### ### Genericidad (``template``)

Permiten crear estructuras y funciones inteligentes independientemente del tipo de dato que almacenen (o

```cpp

template <class T>

class Caja {

 T contenido;

public:

 Caja(T x) { contenido = x; }

```
};  
// Uso: Caja<int> entero(5); Caja<string> texto("Hola");
```

1.4 Compilación con Makefile

Para gestionar proyectos eficientemente con muchos ficheros (`main.cc`, `Punto.hpp`, `Punto.cpp`), delegamos la tarea a un archivo de comandos **Makefile**. Una vez programado, ejecutar simplemente **make** unificará los archivos `.o` solo si han sufrido cambios recientes.

```
“makefile [Makefile] CXX = g++ CXXFLAGS = -Wall -std=c++17
```

```
program: main.o Punto.o $(CXX) -o program main.o Punto.o
```

```
main.o: main.cc Punto.hpp $(CXX) $(CXXFLAGS) -c main.cc “
```

1.5 Por si no te gusta leer teoría

Este simulador contiene un proyecto entero con clases, métodos, `this`, `inline`, `static` y un **Makefile**.

[Simulació interactiva disponible a la web]